The Journal of Open Source Software

LitDet: An AutoML Tool for Accelerating 2D Object Detection Experiments

Loic Tetrel [1¶], **Thomas Checchin** [1,2], **and Louis Pagnier**[1]

1 Kitware SAS, France **2** Université Lumière Lyon 2, Université Claude Bernard Lyon 1, ERIC, 69007, Lyon, France ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Richard Liu

Reviewers:

- @JiaoMaWHU
- @dosvk
- @showdawn

Submitted: 10 June 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Driven by the recent growth of computer vision applications and the increasing complexity of object detection workflows, we developed `LitDet`. `LitDet` is an open-source, domain-agnostic AutoML Python framework designed to significantly accelerate the training, evaluation, and deployment of 2D object detection models thanks to its configuration-driven design.

Statement of need

The rapid evolution of deep learning for computer vision tasks since the advent of ImageNet (Krizhevsky et al., 2012) has yielded powerful foundational libraries, yet a significant operational gap remains for computer vision scientists. Developing 2D object detection pipelines typically requires writing lots of redundant, error-prone boilerplate code. Today, the manual re-coding of foundational modules such as specialized data-loaders, complex training/validation loops, custom evaluation metrics or model exporters should no longer be a necessity. Furthermore, managing machine learning experimentation is still heavily manual. Developers often have to build command-line interface (CLI) from scratch and run multiple experiments (e.g., for hyperparameter optimization).

`LitDet` was designed to solve these friction points by providing a domain-agnostic Automated Machine Learning (AutoML) (Hutter et al., 2019) framework that accelerates the end-to-end development of 2D object detection models. `LitDet` unifies the entire machine-learning lifecycle, from dataset preparation to prediction, with a high-level CLI that abstracts away all core structural modules.

State of the field

In the broader machine learning landscape, `PyTorch` (Paszke et al., 2019; TorchVision maintainers and contributors, 2016) has become the dominant library within the scientific community due to its flexible, imperative style API, and Hugging Face's `Transformers` (Wolf et al., 2020) inherited from it to democratize attention-based deep learning architectures.

To abstract the underlying complexity of these ecosystems and simplify the development of object detection workflows, multiple AutoML solutions (Hutter et al., 2019) exist. Cloud platforms such as `Roboflow` (Dwyer et al., 2026) or `Neptune AI` (Neptune team, 2019) offer robust MLOps (Kreuzberger et al., 2023) capabilities, but their core features are closed-source, not runnable locally, gated behind commercial paywalls, and lack the customization required for reproducible scientific research. Meanwhile, Ultralytics (Jocher et al., 2023) successfully lowered the barrier to entry by introducing an open-source Python package that enables users to train YOLO-family models with minimal setup. However, the codebase is distributed under

DRAFT

an AGPL-3.0 license, significantly restricting its commercial application. While open-source Python alternatives like `MMDetection` (Chen et al., 2019) and `Detectron2` (Wu et al., 2019) allow local configuration, execution, with an impressive set of state-of-the-art models, the integration of novel features (e.g., models) remains highly challenging, often requiring deep expertise of their code. Furthermore, official release cycles for both frameworks have stalled at the time of writing.

Built upon the open-source lightning-hydra-template (Dobler & authors, 2021), `LitDet` was designed from the ground up to be based entirely on a permissive, open-source stack. To achieve this, we integrated established Python libraries, `PyTorch Lightning` (Falcon & The PyTorch Lightning team, 2019) for configurable, tracked, scalable, reproducible training pipelines with experiment tracking integration (Zaharia et al., 2018) and `Hydra` (Yadan, 2019) for flexible, hierarchical configuration management. By adhering strictly to the traditional COCO data format (Lin et al., 2014), `LitDet` seamlessly integrates into existing computer vision workflows without requiring users to adopt platform-locked data specifications. In addition, the software's long-term maintenance and viability are secured by Kitware, that already has a long-standing experience of maintaining and building a business model around open source software, such as `CMake` or `VTK` (Schroeder et al., 2006).

Software design

In order to accelerate the development of 2D computer vision applications, LitDet unifies every phase of the machine learning lifecycle. Its architecture prioritizes customizability, allowing it to seamlessly adapt across diverse scientific and application domains. It is realized by coupling `Hydra` (Yadan, 2019) for configuration management along with `PyTorch Lightning` (Falcon & The PyTorch Lightning team, 2019) to eliminate redundant or boilerplate code, and automate scaling across CPU, GPU, and distributed environments.

The typical `LitDet` workflow follows a standard machine learning workflow, as described in our online documentation: dataset standardization, data loading, model training, model serialization, and downstream evaluation or inference execution. This lifecycle is exposed to the user via three high-level CLI commands: *light-train*, *light-eval*, and *light-predict*.

Input datasets must comply with the standard COCO directory structure and annotation format (Lin et al., 2014). Each runtime execution is orchestrated by composition-driven YAML modules adhering to `Hydra` syntax, which define data parameters, architectural layouts, and runtime stages. This declarative composition dynamically instantiates core `Lightning` components, structurally decoupling data processing pipelines, neural architectures, and training execution loops. The primary design components are structured as follows:

Data processing: To ensure runtime integrity and avoid downstream pipeline failures, an initial data-validation and preprocessing layer applies structural transformations such as box coordinate normalization and label type casting, alongside configurable data augmentation policies (for instance large-scale jittering and horizontal flips (Ghiasi et al., 2020)).

Task workflow: `LitDet` automates the classical machine learning loop for a 2D object detection task by translating abstract task definitions into a fully configured execution pipeline. It automatically orchestrates and instantiates the deep learning model, dataloader, hardware accelerators, custom callbacks, and evaluation metrics. The underlying loss functions are structurally bound to the chosen model, for example a standard 2D detector inherently pairs localization regression (Smooth L1, ...) with classification (cross-entropy, ...) to compute gradients. This iterative fitting loop is driven by the `PyTorch Lightning Trainer`, which handles distributed execution and scheduling dynamics, and is continuously validated against domain-standard metrics such as Mean Average Precision (mAP), F1 score, precision-recall curves, and confusion matrices.

Stage logic: Execution is adapted to the specific needs of each lifecycle stage. While the

DRAFT

training stage optimizes model convergence with learning-rate schedulers, the evaluation and prediction stages shift priority toward computation of accuracy metrics, inference and output visualization. `LitDet` natively embeds specialized logging and experiment-tracking adapters, including `Aim` (Arakelyan et al., 2020) or `MLflow`(Zaharia et al., 2018), alongside visualization and writer callbacks to render and serialize bounding box predictions for qualitative assessment.

All pipeline parameters are supplied with production-ready default YAML configurations. Users can override these parameters directly through CLI arguments or by defining their own experiment YAML configuration, bypassing the need to modify source code. This layout enhances experiment legibility, maintainability, and reproducibility. A representative customized experiment file is structured as follows:

```yaml
# @package _global_

# to execute this experiment run:
# light-train +experiment=detect_example
# (with detect_example.yaml under a directory named "experiment"

defaults:
  - override /data: coco
  - override /task: detect
  - override /task/model: faster_rcnn
  - override /callbacks: [default, prediction_visualizer] # Add callbacks
  - override /trainer: default
  - override /logger: [aim, csv, tensorboard] # Choose your loggers

# all parameters below will be merged with parameters
# from the default configurations set above
# this allows you to overwrite only specified parameters

tags: ["detection", "coco", "fasterrcnn_resnet50_fpn_v2"]

seed: 12345

trainer:
  max_epochs: 12

data:
  is_contiguous: False # coco2017 includes 10 un-annotated categories
  include_background: False # coco2017 does not include backgroud label
  train_transforms:
    # Rewrite all your transformation
    - _target_: torchvision.transforms.v2.RandomHorizontalFlip
    - _target_: torchvision.transforms.v2.ScaleJitter
      target_size: [1024, 1024]

task:
  model:
    # Use Faster R-CNN pre-trained weights
    weights:
      _target_: hydra.utils.get_object
      path: torchvision.models.detection.FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT
    # Change the number of classes accordingly to your dataset
    num_classes: 81
  # use a lr scheduler like reduce_on_plateau
  lr_scheduler_config:
```

DRAFT

```yaml
monitor: "train/loss"
interval: "step"
frequency: 1
strict: True,
name: None,
scheduler:
  _target_: torch.optim.lr_scheduler.ReduceLROnPlateau
  _partial_: true
  mode: min
  factor: 0.1
  patience: 10000
```

Figure 1 maps the end-to-end software pipeline of `LitDet`, tracing the data flow from `Hydra` YAML configurations through to the training, evaluation, and prediction stages executed by `PyTorch Lightning`.

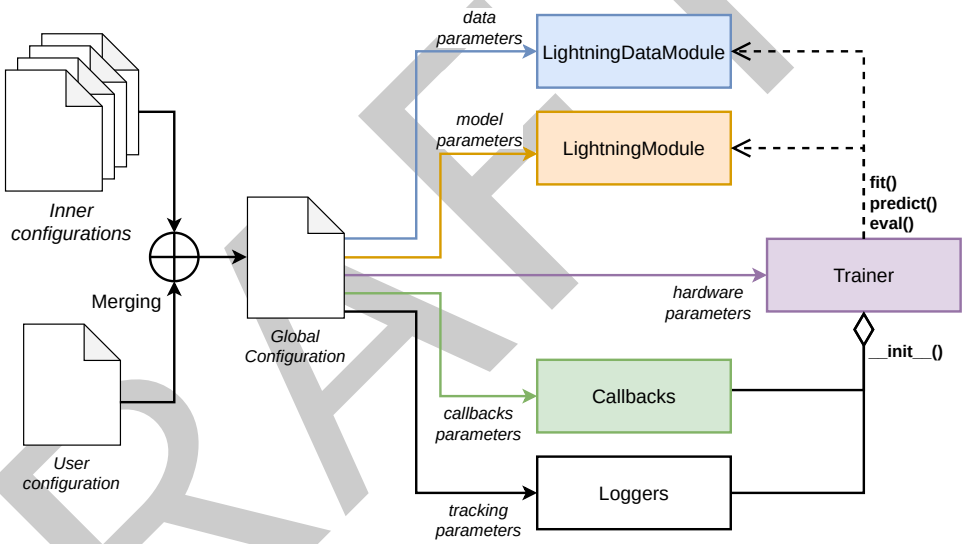


Figure 1: Global workflow of `LitDet`

To ensure straightforward deployment and guarantee environment reproducibility, `LitDet` is distributed as a public PyPI package and an official `Docker` container image (Merkel, 2014). For high-performance computing environments where root administrative privileges are restricted, the `Docker` image can be seamlessly converted into an `Apptainer` container (Kurtzer et al., 2021), maintaining native and reproducible execution performance across distributed clusters.

Research impact statement

`LitDet` has already demonstrated domain-specific utility through its integration within the VIAME toolkit (Dawkins et al., 2017), an open-source platform widely used in marine biology for underwater video analytics and wildlife population assessments and supported by multiple government agencies such as Ifremer and NOAA. Ongoing initiatives are expanding the software's ecosystem into computational medicine. Specifically, `LitDet` is currently being integrated into the DIVE web platform to benchmark real-time object detection models for endoscopic and minimally invasive surgical procedures.

DRAFT

AI usage disclosure

No generative AI tools were used in the design or development of this software. The Gemini (Google) large language model was used solely to assist with documentation writing and the stylistic refinement of this manuscript, with all the final content carefully reviewed by the authors.

Acknowledgements

This work was funded and developed entirely through the internal research and development budget of Kitware SAS.

References

Arakelyan, G., Soghomonyan, G., & The Aim team. (2020). *Aim* (Version 3.9.3). https://doi.org/10.5281/zenodo.6536395

Chen, K., Wang, J., Pang, J., Cao, Y., Xiong, Y., Li, X., Sun, S., Feng, W., Liu, Z., Xu, J., Zhang, Z., Cheng, D., Zhu, C., Cheng, T., Zhao, Q., Li, B., Lu, X., Zhu, R., Wu, Y., … Lin, D. (2019). MMDetection: Open MMLab detection toolbox and benchmark. *arXiv Preprint arXiv:1906.07155*. https://doi.org/10.48550/arXiv.1906.07155

Dawkins, M., Sherrill, L., Fieldhouse, K., Hoogs, A., Richards, B., Zhang, D., Prasad, L., Williams, K., Lauffenburger, N., & Wang, G. (2017). An open-source platform for underwater image and video analytics. *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 898–906. https://doi.org/10.1109/WACV.2017.105

Dobler, K., & authors, template. (2021). Lightning-Hydra-Template. In *GitHub repository*. https://github.com/ashleve/lightning-hydra-template; GitHub.

Dwyer, B., Nelson, J., Hansen, T., & Solawetz, J. (2026). *Roboflow (version 1.0) [software]*. https://roboflow.com

Falcon, W., & The PyTorch Lightning team. (2019). *PyTorch Lightning* (Version 1.4). https://doi.org/10.5281/zenodo.3828935

Ghiasi, G., Cui, Y., Srinivas, A., Qian, R., Lin, T.-Y., Cubuk, E. D., Le, Q., & Zoph, B. (2020). Simple copy-paste is a strong data augmentation method for instance segmentation. arXiv. *arXiv Preprint arXiv:2012.07177*. https://doi.org/10.48550/arXiv.2012.07177

Hutter, F., Kotthoff, L., & Vanschoren, J. (2019). *Automated machine learning: Methods, systems, challenges*. Springer. https://doi.org/10.1007/978-3-030-05318-5

Jocher, G., Qiu, J., & Chaurasia, A. (2023). Ultralytics. In *GitHub repository*. https://github.com/ultralytics/ultralytics; GitHub.

Kreuzberger, D., Kühl, N., & Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, *11*, 31866–31879. https://doi.org/10.1109/ACCESS.2023.3262138

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, *25*. https://doi.org/10.1145/3065386

Kurtzer, G. M., cclerget, Bauer, M., Kaneshiro, I., Trudgian, D., & Godlove, D. (2021). *Hpcng/singularity: Singularity 3.7.3* (Version v3.7.3). Zenodo. https://doi.org/10.5281/zenodo.4667718

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick,

DRAFT

C. L. (2014). Microsoft coco: Common objects in context. *European Conference on Computer Vision*, 740–755. https://doi.org/10.1007/978-3-319-10602-1_48

Merkel, D. (2014). Docker: Lightweight linux containers for consistent development and deployment. *Linux J.*, *2014*(239). https://doi.org/10.5555/2600239.2600241

Neptune team. (2019). *neptune.ai*. https://neptune.ai/

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., & others. (2019). Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, *32*. https://doi.org/10.48550/arXiv.1912.01703

Schroeder, W., Martin, K., & Lorensen, B. (2006). *The visualization toolkit (4th ed.)*. Kitware. ISBN: 978-1-930934-19-1

TorchVision maintainers and contributors. (2016). *TorchVision: PyTorch's Computer Vision library*. https://github.com/pytorch/vision

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., Platen, P. von, Ma, C., Jernite, Y., Plu, J., Xu, C., Scao, T. L., Gugger, S., … Rush, A. M. (2020). *HuggingFace's transformers: State-of-the-art natural language processing*. https://doi.org/10.18653/v1/2020.emnlp-demos.6

Wu, Y., Kirillov, A., Massa, F., Lo, W.-Y., & Girshick, R. (2019). *Detectron2*. https://github.com/facebookresearch/detectron2.

Yadan, O. (2019). *Hydra - a framework for elegantly configuring complex applications*. Github. https://github.com/hydra-ecosystem/hydra

Zaharia, M. A., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Eng. Bull.*, *41*, 39–45. https://api.semanticscholar.org/CorpusID:83459546

DRAFT